

JVM Memory Analysis — Practical Guide

A hands-on POC for understanding heap memory behaviour in a Spring Boot application using VisualVM, IntelliJ, and Spring Boot Actuator endpoints.

Table of Contents

- 1. [Why Monitor JVM Memory?](#)
- 2. [Tooling Options](#)
- 3. [Project Setup](#)
- 4. [The Memory Stress Controller](#)
- 5. [Endpoints Reference](#)
- 6. [Step-by-Step Experiment](#)
- 7. [Reading the Heap Dump](#)
- 8. [Finding Your Object in the Heap](#)
- 9. [Preventing OutOfMemoryError — Best Practices](#)
- 10. [Key Takeaways](#)

1. Why Monitor JVM Memory?

JVM memory issues are among the most common causes of production incidents. Left undetected, they lead to:

Symptom	Root Cause
OutOfMemoryError: Java heap space	Objects accumulate faster than GC can collect
Slow response times	GC runs too frequently (GC pressure)
Application crash / restart loop	Heap exhausted, JVM cannot recover
Memory leak	Objects stay referenced and are never collected

Monitoring memory *before* a crash lets you identify the problem early and fix it proactively.

2. Tooling Options

Option A — VisualVM (Standalone)

A free, powerful profiler that connects to any running JVM.

- Download: <https://visualvm.github.io/download.html>
- Capabilities: Heap monitoring, thread analysis, heap dumps, CPU profiling

Option B — VisualVM via IntelliJ Plugin

Run VisualVM directly from IntelliJ without switching windows.

- Plugin: [VisualVM Launcher — JetBrains Marketplace](#)
- Install via: Settings → Plugins → search "VisualVM Launcher"

Option C — Spring Boot Actuator Endpoints

Lightweight, no external tool needed. Exposes memory metrics over HTTP.

Requires `spring-boot-starter-actuator` dependency — see [Section 3](#).

3. Project Setup

Maven Dependency

Add Actuator to `pom.xml` :

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Expose Actuator Endpoints

In `application.properties` :

```
management.endpoints.web.exposure.include=health,metrics,heapdump  
management.endpoint.heapdump.enabled=true
```

4. The Memory Stress Controller

This controller intentionally simulates memory pressure so you can observe heap behaviour in real time.

```

@RestController
@RequestMapping("/memory")
public class MemoryStressController {

    // Bean-scoped fields – live as long as the Spring bean lives
    private final List<byte[]> memoryHolder = new ArrayList<>();
    private final Map<String, byte[]> hashMapHolder = new HashMap<>();

    // Spawns 2 threads that each allocate 100MB
    @GetMapping("/thread-stress")
    public String threadStress() throws InterruptedException {

        // Thread 1 – fills an ArrayList with 1MB byte arrays
        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 100; i++) {
                memoryHolder.add(new byte[1024 * 1024]); // 1MB each → 100MB total
                try { Thread.sleep(50); } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
        });

        // Thread 2 – fills a HashMap with 1MB byte arrays
        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 100; i++) {
                hashMapHolder.put("key-" + i, new byte[1024 * 1024]); // 1MB each → 100MB total
                try { Thread.sleep(50); } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
        });

        t1.start();
        t2.start();
        return "2 threads started - watch heap in VisualVM!";
    }

    // Releases all references and hints GC
    @GetMapping("/clear")
    public String clear() {
        memoryHolder.clear();
        hashMapHolder.clear();
        for (int i = 0; i < 3; i++) {

```

```

        System.gc();
        try { Thread.sleep(500); } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
    long usedAfter = (Runtime.getRuntime().totalMemory()
        - Runtime.getRuntime().freeMemory()) / 1024 / 1024;
    return "Memory cleared! Used memory now: " + usedAfter + " MB";
}

// Returns live memory stats
@GetMapping("/stats")
public Map<String, String> stats() {
    Runtime rt = Runtime.getRuntime();
    Map<String, String> stats = new HashMap<>();
    stats.put("usedMemory", ((rt.totalMemory() - rt.freeMemory()) / 1024 / 1024) + " MB");
    stats.put("freeMemory", (rt.freeMemory() / 1024 / 1024) + " MB");
    stats.put("totalMemory", (rt.totalMemory() / 1024 / 1024) + " MB");
    stats.put("maxMemory", (rt.maxMemory() / 1024 / 1024) + " MB");
    stats.put("listSize", memoryHolder.size() + " entries");
    stats.put("hashMapSize", hashMapHolder.size() + " entries");
    return stats;
}
}

```

How the Allocation Works

Loop: 100 iterations × 1MB = 100MB per collection

Thread 1 (ArrayList):	Thread 2 (HashMap):
memoryHolder[0] → 1MB	key-0 → 1MB
memoryHolder[1] → 1MB	key-1 → 1MB
...	...
memoryHolder[99] → 1MB	key-99 → 1MB
Total: 100MB	Total: 100MB

Grand Total: ~200MB allocated

Thread.sleep(50) between iterations makes the heap grow *gradually* — so you can watch it climb in VisualVM in real time rather than all at once.

5. Endpoints Reference

Endpoint	Description
GET /memory/thread-stress	Start 2 threads allocating ~200MB total
GET /memory/stats	View live heap usage and collection sizes
GET /memory/clear	Release all references and suggest GC
GET /actuator/metrics/jvm.memory.used	JVM used memory via Actuator
GET /actuator/metrics/jvm.memory.max	JVM max memory via Actuator
GET /actuator/heapdump	Download a .hprof heap dump file

6. Step-by-Step Experiment

Step 1 — Baseline: Check Memory Before Stress

Before triggering any load, call `/actuator/metrics/jvm.memory.used` to record your baseline. The response breaks down memory by area (`heap` , `nonheap`) and by memory pool (`G1 Old Gen` , `G1 Eden Space` , `Metaspace` , etc.).

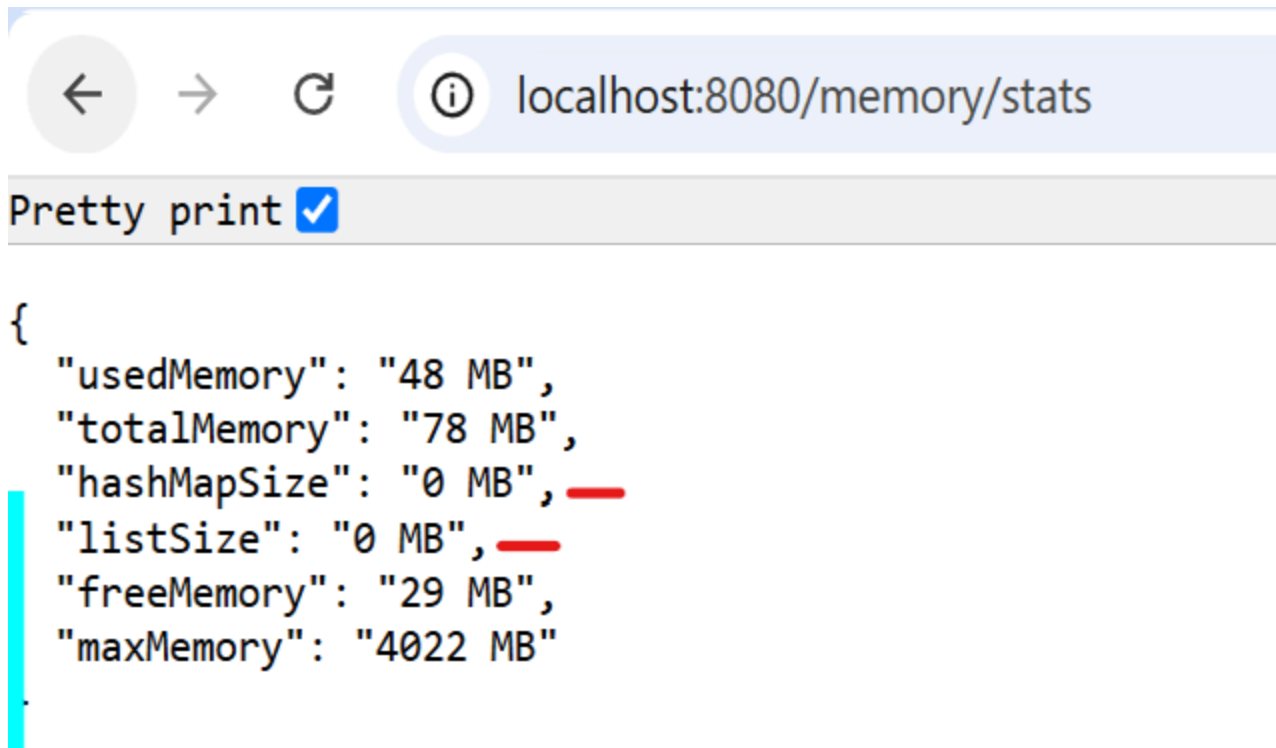
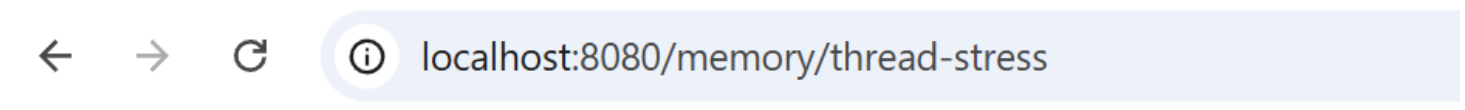


Fig 1 — Actuator `jvm.memory.used` before any stress is applied. Value: ~194MB baseline.

Step 2 — Trigger Memory Stress

Call `GET /memory/thread-stress`. Both threads start immediately. Thread 1 fills the `ArrayList` and Thread 2 fills the `HashMap`, each allocating 1MB every 50ms across 100 iterations (~200MB total).



2 threads started - watch heap in VisualVM!

Fig 2 — Response from `/memory/thread-stress`. Switch to VisualVM immediately to watch heap climb.

Step 3 — Observe Heap Growth in VisualVM

The VisualVM Monitor tab shows the heap climbing from ~450MB toward ~750MB as both threads run in parallel. The sharp drop points (marked in red at 12:40 and 12:50) show exactly where `/clear` was called and GC collected the released objects.

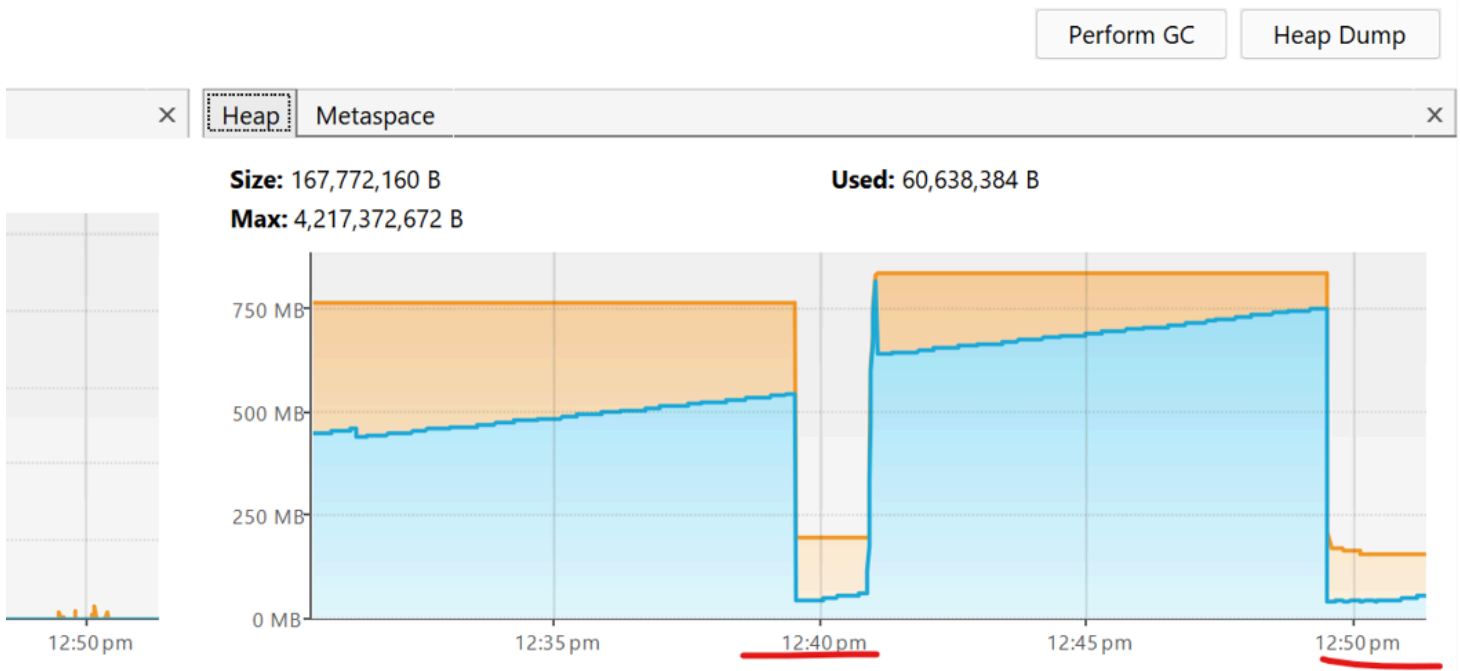


Fig 3 — VisualVM heap graph: blue = used heap, orange = allocated heap size. Each ramp is one `/thread-stress` call; each drop is one `/clear` call.

Step 4 — Check Stats During Stress

While threads are running (or just after they finish), call `/memory/stats` . The response shows `hashCode` at 100 entries and `listSize` at 170 entries (threads may still be running), with `usedMemory` at 705MB — nearly the full allocated heap.



```
{
  "availableTags": [
    {
      "tag": "area",
      "values": [
        "heap",
        "nonheap"
      ]
    },
    {
      "tag": "id",
      "values": [
        "CodeHeap 'profiled nmethods'",
        "G1 Old Gen",
        "CodeHeap 'non-profiled nmethods'",
        "G1 Survivor Space",
        "Compressed Class Space",
        "Metaspace",
        "G1 Eden Space",
        "CodeHeap 'non-nmethods'"
      ]
    }
  ],
  "baseUnit": "bytes",
  "description": "The amount of used memory",
  "measurements": [
    {
      "statistic": "VALUE",
      "value": 194564704
    }
  ],
  "name": "jvm.memory.used"
}
```

Fig 4 — /memory/stats during stress: hashMapSize = 100MB, listSize = 170MB, usedMemory = 705MB. Only 12MB free.

Step 5 — Take a Heap Dump Before Clearing

While memory is still high, capture a heap dump to inspect what is holding the memory:

- **VisualVM:** Click the **Heap Dump** button in the Monitor tab

- **Actuator:** GET /actuator/heapdump → downloads a .hprof file

Open the dump → go to **Objects** view → search for `MemoryStressController`. Expand the instance fields — you will see `memoryHolder` (`ArrayList` with 100 elements) and `hashMapHolder` (`HashMap` with 100 entries).

Name	Count	Size	Retained (sort to get)
com.example.securityExample.controller.MemoryStressController	1 (0%)	24 B (0%)	n/a
com.example.securityExample.controller.MemoryStressController#1		24 B (0%)	n/a
<fields>			
memoryHolder = java.util.ArrayList#5251 : 100 elements		24 B (0%)	n/a
hashMapHolder = java.util.HashMap#2406 : 100 elements		48 B (0%)	n/a
static <resolved_references> = java.lang.Object[]#8523 : 31 items		144 B (0%)	n/a
static <classLoader> = org.springframework.boot.devtools.restart.classloader.RestartClassLoader#1		88 B (0%)	n/a
<references>			
com.example.securityExample.controller.MemoryStressController\$\$Lambda+0x0000000020b44ad8	0 (0%)	0 B (0%)	n/a
com.example.securityExample.controller.MemoryStressController\$\$Lambda+0x0000000020b44ab8	0 (0%)	0 B (0%)	n/a

Fig 5 — Heap dump before clear: `memoryHolder` → `ArrayList` with 100 elements, `hashMapHolder` → `HashMap` with 100 entries. These 200 objects hold ~200MB on the heap.

Step 6 — ArrayList Instances in the Heap Dump

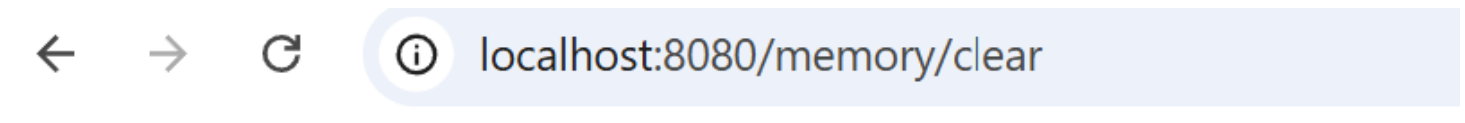
When you expand the heap dump Objects view, you can see individual `java.util.ArrayList` instances numbered sequentially (`ArrayList#84` through `ArrayList#100`). These represent the tracked allocations visible in the profiler.

Name	Count	Size	Retained (sort to get)
java.util.ArrayList#84 : 1 element		24 B (0%)	n/a
java.util.ArrayList#85 : 1 element		24 B (0%)	n/a
java.util.ArrayList#86 : 1 element		24 B (0%)	n/a
java.util.ArrayList#87 : 1 element		24 B (0%)	n/a
java.util.ArrayList#88 : 1 element		24 B (0%)	n/a
java.util.ArrayList#89 : 1 element		24 B (0%)	n/a
java.util.ArrayList#90 : 1 element		24 B (0%)	n/a
java.util.ArrayList#91 : 1 element		24 B (0%)	n/a
java.util.ArrayList#92 : 1 element		24 B (0%)	n/a
java.util.ArrayList#93 : 1 element		24 B (0%)	n/a
java.util.ArrayList#94 : 1 element		24 B (0%)	n/a
java.util.ArrayList#95 : 1 element		24 B (0%)	n/a
java.util.ArrayList#96 : 1 element		24 B (0%)	n/a
java.util.ArrayList#97 : 1 element		24 B (0%)	n/a
java.util.ArrayList#98 : 1 element		24 B (0%)	n/a
java.util.ArrayList#99 : 1 element		24 B (0%)	n/a
java.util.ArrayList#100 : 1 element		24 B (0%)	n/a

Fig 6 — ArrayList instances in heap dump. Each numbered instance (ArrayList#84 to ArrayList#100) is 24B individually, but the byte arrays they reference total ~100MB.

Step 7 — Clear Memory

Call `GET /memory/clear`. The endpoint calls `.clear()` on both collections (removing all references to the byte arrays), then calls `System.gc()` three times to encourage the JVM to collect the now-unreachable objects.



Memory cleared!

Fig 7 — Response from `/memory/clear` confirms both collections have been emptied and GC was requested.

Step 8 — Confirm via Stats After Clear

Call `/memory/stats` again immediately after clearing. Both `hashMapSize` and `listSize` return `0`, and `usedMemory` drops from 705MB back to 48MB — close to the original baseline.

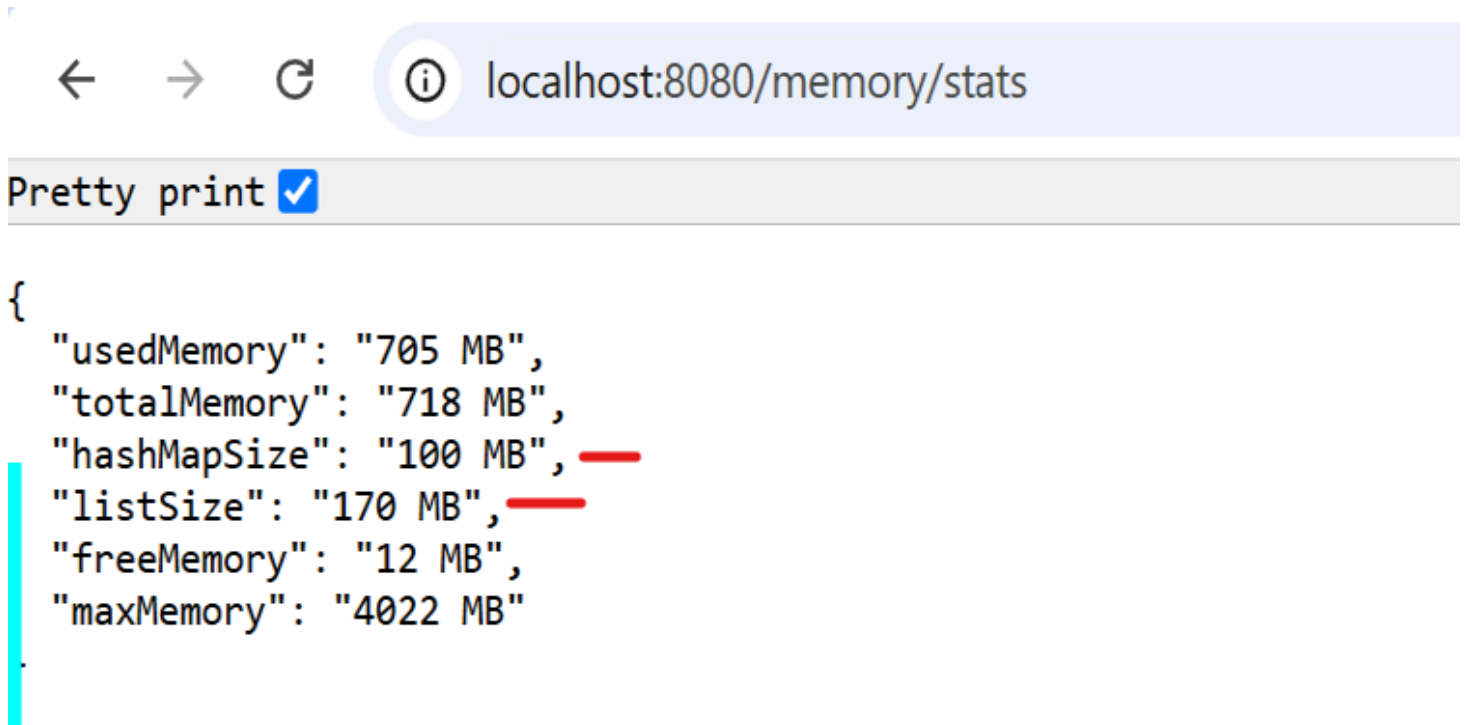


Fig 8 — /memory/stats after clear: hashMapSize = 0, listSize = 0, usedMemory = 48MB. The ~200MB has been released and collected.

Step 9 — Confirm in Heap Dump After Clear

Take a second heap dump after clearing. Expand MemoryStressController#1 → <fields> . Both memoryHolder and hashMapHolder now show **0 elements** — the byte arrays are unreachable and have been collected by GC.

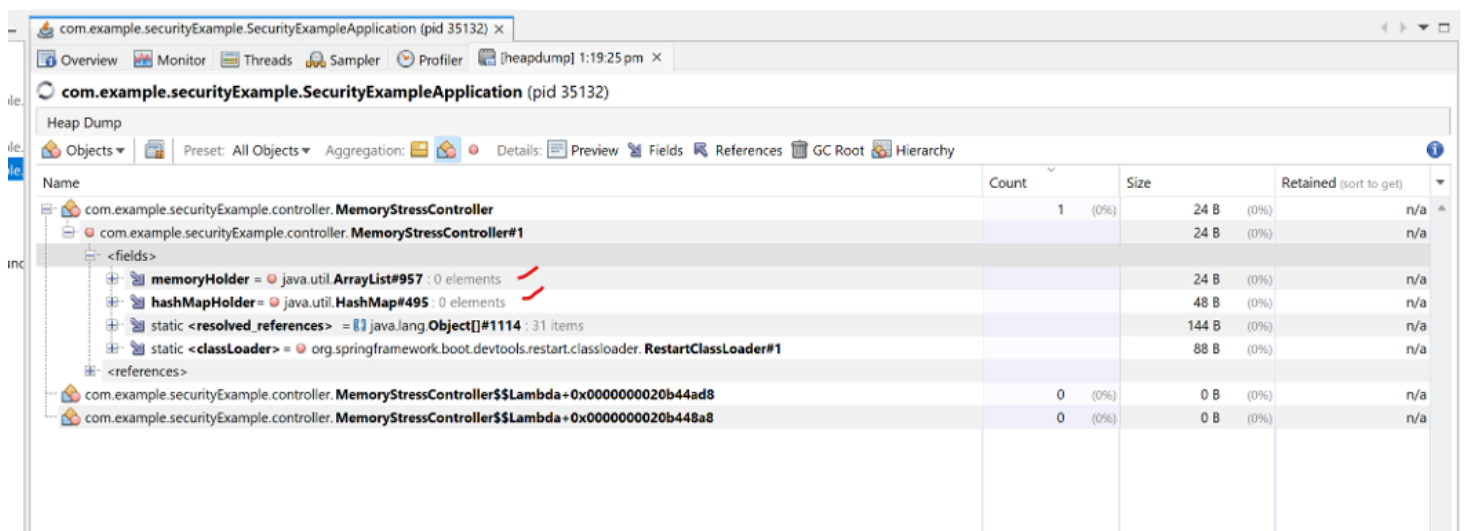


Fig 9 — Heap dump after clear: memoryHolder = 0 elements, hashMapHolder = 0 elements. The Spring bean still exists as a singleton, but holds no data.

7. Reading the Heap Dump

When you open the heap dump in VisualVM (Objects tab), the top entries by count are typically JVM and framework internals — this is expected and normal.

Class	What it is
<code>java.util.LinkedHashMap\$Entry</code>	Spring bean registry and session maps
<code>java.lang.Object[]</code>	Backing arrays for collections
<code>java.util.ArrayList</code>	Framework lists and your <code>memoryHolder</code>
<code>byte[]</code>	Your allocated data — 100 entries of 1MB each

Focus on Retained size, not just count. A class with 100 instances but 100MB retained is far more important than one with 10,000 instances at 24B each.

Identifying a Real Leak

Look for your own application classes in the list:

- `com.example.securityExample.*` appearing with large retained sizes
- Object counts that grow across multiple heap dumps taken over time
- Objects that should have been released but still show live references

8. Finding Your Object in the Heap

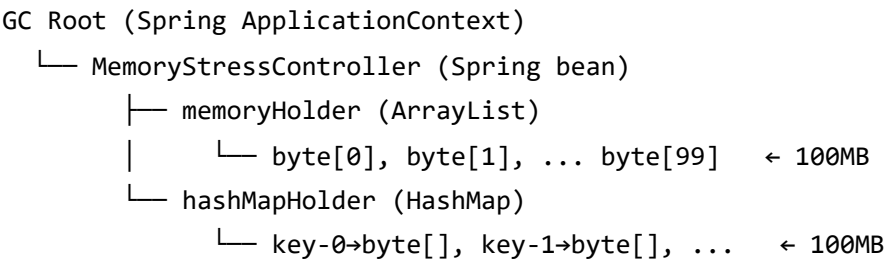
To locate `memoryHolder` specifically inside VisualVM:

- Open the heap dump → **Objects** tab → type `MemoryStressController` in the filter
- You will see **1 instance** — the Spring singleton bean
- Expand it → you will see two fields:

```
memoryHolder → ArrayList (elementData → byte[0] ... byte[99])
hashMapHolder → HashMap (table → Entry[key-0] ... Entry[key-99])
```

- Right-click the instance → **Show in GC Roots**

The GC Root chain explains why memory cannot be released until `.clear()` is called:



As long as the Spring bean is alive, it acts as a GC root — and everything it references stays on the heap.

9. Preventing OutOfMemoryError — Best Practices

Code-Level Practices

Practice	Why it matters
Avoid unbounded collections as instance or static fields	They accumulate indefinitely — prefer local variables or bounded caches
Use <code>WeakReference</code> or <code>SoftReference</code> for caches	JVM can collect these under memory pressure
Stream large data instead of loading it into memory	Never load an entire file or result set into a <code>List</code>
Apply capacity limits and eviction policies	Use <code>LinkedHashMap</code> with <code>removeEldestEntry()</code> or Caffeine/Guava cache
Always close resources with try-with-resources	Prevents off-heap (native) memory leaks

JVM Configuration

```
# Set explicit heap bounds – never leave max heap open-ended
-Xms512m                # Initial heap size
-Xmx1g                  # Maximum heap size

# Enable GC logging to detect pressure early
-Xlog:gc*:file=gc.log:time,uptime

# Auto-generate heap dump on OOM for post-mortem analysis
-XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=/var/log/heapdump.hprof
```

Monitoring Signals — Act Before It Crashes

Signal	Threshold to investigate
Heap used / max consistently above 80%	Memory pressure — likely leak or undersized heap
GC time above 5% of total CPU	GC is becoming a performance bottleneck
Old Gen keeps growing between GC cycles	Classic memory leak pattern
<code>java.lang.OutOfMemoryError</code> in logs	Already too late — monitor the signals above first

Spring Boot Actuator Monitoring

Track these metrics in Prometheus, Grafana, or any APM tool:

```
jvm.memory.used{area="heap"} > 80% of jvm.memory.max{area="heap"}
jvm.gc.pause{action="end of major GC"} – watch frequency and duration
```

10. Key Takeaways

- **memoryHolder and hashMapHolder are Spring bean fields.** They live for the entire application lifetime. Any data added stays in memory until explicitly removed via `.clear()`.
- **`.clear()` removes references, not the containers.** The `byte[]` objects become unreachable and GC collects them. The `ArrayList` and `HashMap` themselves remain as fields of the singleton

bean.

- **Thousands of ArrayLists in the heap is normal.** Spring, Hibernate, and the servlet container use hundreds of internal lists. Focus on retained size and look for your own classes.
- **Heap dumps are your best diagnostic tool.** Always take one while the problem is active — before restarting the JVM — so you have evidence to analyse.
- **Prevention is better than recovery.** Configure `-XX:+HeapDumpOnOutOfMemoryError`, set explicit heap bounds with `-Xmx`, and continuously monitor `jvm.memory.used` against the heap max.

Conclusion : Why instantly we could not see the `clear()` impact

when `clear()` is called

↓

`memoryHolder.clear()` → removes references from List

`hashMapHolder.clear()` → removes references from HashMap

↓

`byte[]` objects now have NO references ← eligible for GC

↓

BUT still sitting in heap until GC runs

↓

Heap Dump taken here ← still shows 100 items ❌

↓

Perform GC clicked in VisualVM

↓

`byte[]` objects collected and removed from heap

↓

Heap Dump taken here ← shows 0 items ✅

POC built with Spring Boot · Profiled with VisualVM · Monitored via Spring Actuator